

Understanding Synthesizable Verilog 😊



IIT KHARAGPUR



NPTEL ONLINE
CERTIFICATION COURSES

Lecture 28: SYNTHESIZABLE VERILOG

PROF. INDRANIL SENGUPTA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

About the Verilog Language

- The language Verilog has a large number of features, most of which are supported by the simulation tools.
- Unfortunately, several of the language constructs are not supported by synthesis tools.
 - The language subset that can be synthesized is known as “*Synthesizable Verilog*” subset.
- Here we shall state the language features not supported by most of the synthesis tools.
 - Best be avoided if the objective is to map the design to hardware.



IIT KHARAGPUR



NPTEL ONLINE
CERTIFICATION COURSES

Hardware Modeling Using Verilog



Synthesis Rules for Combinational Logic

- The output of a combinational logic circuit at time t should depend only upon the inputs applied at time t .
- Rules to be followed:
 - Avoid technology dependent modeling (i.e. implement functionality, not timing).
 - There must not be any feedback in the combinational circuit.
 - For “*if...else*” or “*case*” constructs, the output of the combinational function must be specified for all possible input cases.
 - If the rules are not followed, the circuit may be synthesized as sequential.



IIT KHARAGPUR



NPTEL ONLINE
CERTIFICATION COURSES

Hardware Modeling Using Verilog



Styles for Synthesizable Combinational Logic

- The possible styles for modeling combinational logic are as follows:
 - Netlist of Verilog built-in primitives like gate instances (AND, OR, NAND, etc.).
 - Combinational UDP (*not all synthesis tools support this*).
 - Continuous assignments.
 - Functions.
 - Behavioral statements.
 - Tasks without event or delay control.
 - Interconnected modules of one or more of the above.



(a) As a Gate Netlist

```
module example (x1, x2, x3, x4, y);  
  input x1, x2, x3, x4;  
  output y;  
  wire w1, w2, w3;  
  or (w1, x1, x2);  
  or (w2, x3, x4);  
  xor (w3, x3, x4);  
  nand (y, w1, w2, w3);  
endmodule
```



(b) Using Continuous Assignment

```
module carry (cout, a, b, c);  
  input a, b, c;  
  output cout;  
  
  assign cout = (a & b) | (b & c) |  
                (c & a);  
endmodule
```

Shall be mapped to gates or cells from some target technology library.
The Boolean equations are optimized during synthesis.



(c) Using Procedural Blocking Assignment

```
module mux2to1 (f, in0, in1, sel);
  input in0, in1, sel;
  output reg f;

  always @(in0 or in1 or sel)
    if (sel) f = in1;
    else    f = in0;
endmodule
```

Inputs to the behavior (*here in0, in1, sel*) must be included in the event control expression; otherwise, a latch will be inferred at the output.



(d) Using Functions in Verilog

```
module fulladder (s, cout, a, b, cin);
  input a, b, cin;
  output s, cout;
  assign s = sum(a, b, cin);
  assign cout = carry(a, b, cin);
endmodule
```

```
function sum;
  input x, y, z;
  begin
    sum = x ^ y ^ z;
  end
```

```
function carry;
  input x, y, z;
  begin
    carry = (x&y) | (y&z) | (z&x);
  end
```



```
module fulladder (s, cout, a, b, cin);
  input a, b, cin;
  output reg s, cout;

  always @(a or b or cin)
    FA (s, cout, a, b, cin);

  task FA;
    output sum, carry;
    input A, B, C;
    begin
      #2 sum = A ^ B ^ C;
      carry = (A&B) | (B&C) | (C&A);
    end
  endtask
endmodule
```

(e) Using Tasks

The arguments must be specified in the same order as they appear in the task declaration.

More than one output value can be returned.



Difference between Function and Task

Function	Task
A function can call another function but not another task.	A task can call other tasks and functions.
A function executes in 0 simulation time.	A task may execute in nonzero simulation time.
A function cannot contain any delay, event, or timing control statement.	A task can contain delay, event, or timing control statements.
A function always return a single value.	A task can pass multiple values through "output" and "inout" type arguments.
A function must have at least one input argument.	A task can have zero or more arguments of type "input", "output", or "inout".



Summary: Synthesizable Verilog Constructs

- *"module ... endmodule"*
- Instantiation of a synthesizable module
- *"always"* construct
- *"assign"* statements
- Built-in gate primitives
- User defined primitives – combinational only
- *"parameter"* statement
- *"functions"* and *"tasks"*
- *"for"* loop
- Almost all operators
- Blocking and non-blocking assignments
- *"if ... else"*, *"case"*, *"casex"* and *"casez"*
- Bits and part select of vectors



Non-Synthesizable Constructs

- *"initial"* construct
- Delays in assignments and test benches.
- *"time"* construct
- *"real"* data type
- The operators *"==="* and *"!=="*
- *"fork ... join"* constructs
- *"force ... release"* constructs
- Variables in loop control

